Desarrollo de Aplicaciones Web

CSS Layouts y Responsividad

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Pablo García Tahoces

CSS. *Diseño Responsivo*.

Layout → Forma en que se disponen los diferentes elementos de una página.

En los primeros años de la web, los diseños del *layout* se hacían para un tamaño de pantalla fijo. En caso de abrir la página con tamaños diferentes aparecían barras de desplazamiento (*scrolls*) o espacios desaprovechados.

A medida que aparecieron dispositivos con tamaños más dispares, el concepto de *Diseño Web Responsivo* (*RWD*) fue creado. Esto se vio reforzado con la aparición de los móviles y los diseños para dichos dispositivos.

La aproximación actual es crear un diseño simple (a una sola columna), pensado para móviles y luego comprobar si se va a abrir con pantallas más grandes, en cuyo caso se aplicará un diseño a varias columnas (*mobile first design*).

CSS. *Diseño Responsivo*.

Layout → Forma en que se disponen los diferentes elementos de una página.

Las dos opciones tradicionales para abordar el diseño del layout eran:

- Layout Fijo.- se fija de forma absoluta el tamaño de las cajas, con independencia del tamaño de la ventana del navegador → Es necesario fijar el ancho del contenido total:

```
width:      800 px;
```

```
margin-left:     auto;
margin-right:    auto;
```

- Layout líquido.- no se fija un tamaño fijo. El navegador adapta el tamaño de los diferentes elementos al tamaño de la ventana vs la cantidad de contenido de los mismos.

CSS. *Diseño Responsivo*.

Layout → Forma en que se disponen los diferentes elementos de una página.

Las opciones actuales son resultado de una evolución:

- *Cameron Adams* (2004) introduce la idea de crear diseños que se adapten al tamaño de la pantalla, usando JavaScript para detectar dicho tamaño y cargar así la versión CSS adecuada

- *Ethan Marcotte* (2010) introduce el término "diseño responsivo" basado en:

  1. Rejillas Fluidas (*fluid grids*) → Contenidos adaptados al tamaño.

  2. Imágenes Fluidas (*fluid images*) → Imágenes adaptadas al tamaño.

  3. Solicitud de tipo de medio (*media query*) → El diseño se adapta al tamaño. Permiten realizar una serie de tests para comprobar qué dispositivo está abriendo la página y aplicar el CSS adecuado.

CSS3. *Diseño Responsivo*.

Diseños responsivos → Permiten construir una página web, que se adapta a diferentes tamaños de dispositivos.

A nivel de web se consigue mediante etiquetas HTML y CSS exclusivamente.

Se utilizarán valores relativos para fijar las dimensiones de los elementos.

viewport → Área de una página web visible para el usuario.

HTML5 permite controlar el *viewport* mediante el uso de etiquetas *<meta>:*

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

width= device-width → Fija el ancho de la página según el dispositivo con el que se abra.

initial-scale= 1.0 → Fija el nivel de zoom inicial.

CSS3. *Diseño Responsivo*.

Reglas *@media* → Permiten definir bloques de reglas según dispositivos.

@media not|only *mediatype* and *(mediafeature* and|or|not *mediafeature)* {
CSS-Code;
}

- not.- revierte el significado de lo que viene a continuación.
- only.- previene a los navegadores que no soportan reglas @media. No tiene efecto en navegadores modernos.
- and.- permite combinar tipos de dispositivos (*mediatype*) y sus características (*mediafeature*).

CSS3. *Diseño Responsivo*.

Reglas *@media* → Permiten definir bloques de reglas según dispositivos.

Las características de los dispositivos que se pueden chequear, son:

- width y height del viewport.
- width y height del dispositivo.
- Orientación (landscape o portrait).
- Resolución.
- …

```
@media screen and (max-width: 1200px) {
    body {
        ……..
    }
}
```

CSS3. *Diseño Responsivo*.

Reglas *@media* → Permiten definir bloques de reglas según dispositivos.

Puntos de ruptura.- dimensiones a partir de las cuales se planifica el cambio de diseño del *layout.* Es posible introducir varios, para diferentes tipos de dispositivos.

Tamaños típicos:

- min-width: 320 px → Móviles de pequeño tamaño
- min-width: 480 px → Pequeños dispositivos y mayoría de móviles
- min-width: 768 px → Mayoría de tablets
- min-width: 992 px → Ordenadores con pantallas pequeñas
- min-width: 1200 px → Ordenadores con pantallas anchas y grandes dispositivos

CSS3. *Diseño Responsivo*.

Reglas *@media* → Permiten definir bloques de reglas según dispositivos.

Puntos de ruptura.- dimensiones a partir de las cuales se planifica el cambio de diseño del *layout.* Es posible introducir varios, para diferentes tipos de dispositivos.

```
<body>
    <h1>Responsividad DAW</h1>
    <p>Este es un ejemplo que muestra el efecto de la responsividad
</body>
```

```
@media only screen and (max-width: 600px) {
  body {
    background-color: black;
    color: white;
  }
}
```

Responsividad DAW

Este es un ejemplo que muestra el efecto de la responsividad.

Responsividad DAW

Este es un ejemplo que muestra el efecto de la responsividad.

CSS3. *Diseño Responsivo*.

Reglas *@media* → Permiten definir bloques de reglas según dispositivos.

Alternativamente se podrían incluir estas *reglas media* como parte de las etiquetas <link> que permiten seleccionar hojas de estilo alternativas.

```
<link … media="screen and (min-width: 900px)" href="pantallaGrande.css">

<link … media="screen and (max-width: 600px)" href="pamtallaPequeña.css">
```

CSS. *Tipografía Responsiva*.

Tipografía Responsiva → Tamaños de fuente adaptados al tamaño del dispositivo.

```html
<body>
    <h1>Tipografía Responsiva</h1>
    <p>Este es un ejemplo de tipografía responsiva</p>
</body>
```

```css
html {
  font-size: 12px;
}
h1 {
  font-size: 2rem;
}
@media (min-width: 650px) {
  h1 {
    font-size: 4rem;
  }
}
p {
  font-size: calc(1rem + 1vw);
}
```

Tipografía Responsiva

Este es un ejemplo de tipografía responsiva

Tipografía Responsiva

Este es un ejemplo de tipografía responsiva

Nota: *rem → em respecto a root (html)*

CSS. *Diseño Responsivo*.

Imágenes responsivas → Imágenes adaptadas “en origen” al tamaño del dispositivo.

Solución tradicional →

```
img {
    max-width: 80%;
}
```

Solución actual →

```
<picture>
  <source media="(min-width:900px)" srcset="miImagen_1.jpg">
  <source media="(min-width:650px)" srcset="miImagen_2.jpg">
  <img src="miImagen_3.jpg">
</picture>
```

Nota → *Esta solución busca ahorrar ancho de banda*

CSS. Flexible Grids.

Mecanismo *convencional* para conseguir responsividad, basado en:

- Uso de float
- Uso de anchos relativos basados en la expresión:

  width= 100 x target / context

  Donde *target* es el ancho para elemento deseado
  context es el ancho de la página

Ej.- width: 31,25%; → 300/960= 0,3125

CSS. Flexible Grids.

```html
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
</head>

<body>

    <div class="contenedor">
      <div class="col1">
        <p>Este diseño es responsivo. Ver que ocurre cuando se reduce el
      </div>
      <div class="col2">
          <p>Lorem ipsum dolor sit amet, consectetuer adipiscing elit, s
          <p>Id sea utamur aperiam, te per choro accusamus consulatu. Br
      </div>
    </div>
</body>
```

```css
@media screen and (min-width: 600px) {
  .col1 {
    width: 31.25%;
    float: left;
  }
  .col2 {
    width: 64.75%;
    float: right;
  }
}
```

Este diseño es responsivo. Ver que ocurre cuando se reduce el ancho del navegador.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi.

Id sea utamur aperiam, te per choro accusamus consulatu. Brute munere corrumpit ut pri. Ea ipsum appareat erroribus mea. Mei probo inani aliquid ad. Omnis fabulas concludaturque an vix, an noluisse takimata facilisis pro, sit te volumus mandamus. Ad malorum imperdiet duo, ea possim utamur accusamus vix.

Este diseño es responsivo. Ver que ocurre cuando se reduce el ancho del navegador.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi.

Id sea utamur aperiam, te per choro accusamus consulatu. Brute munere corrumpit ut pri. Ea ipsum appareat erroribus mea. Mei probo inani aliquid ad. Omnis fabulas concludaturque an vix, an noluisse takimata facilisis pro, sit te volumus mandamus. Ad malorum imperdiet duo, ea possim utamur accusamus vix.

CSS. Tecnologías de layout modernas.

Se trata de métodos que asumen que el diseño que se va a crear es flexible y facilitan su implementación.

Actualmente existen tres tecnologías diferentes:

- Multicol
- Flex Container
- Grid

CSS Multicol

Multicol.- Es el más antiguo de los métodos. Se basa en indicarle al navegador en cuantas columnas se quiere dividir un contenedor y que aquel haga los cálculos en función del tamaño del viewport.

El número de columnas se realiza a través de la propiedad *column-count*

```
<body>

    <h1>Multicol</h1>
    <p>La propiedad column-count define

    <p class="contenedor">
        Lorem ipsum dolor sit amet, cons
    </p>

</body>
```

```
.contenedor {
    column-count: 3;
}
```

Multicol

La propiedad column-count define el número de columnas en que se divide un elemento:

Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum. Typi non habent claritatem insitam; est usus legentis in iis qui facit eorum claritatem. Investigationes demonstraverunt lectores legere me lius quod ii legunt saepius.

CSS Flex Container

Flex Container.- Mecanismo de CSS diseñado para crear un contenedor flexible que facilita la disposición de elementos en una dimensión, ya sea en forma de filas o columnas.

El uso de grid se realiza a través de la propiedad *display.*

Para el uso de este mecanismo se definen propiedades tanto para el *contenedor*, como para los ítems que están en su interior.

El contenedor se crea mediante display: flex;

Los ítems se pueden crear en términos de filas o columnas.

CSS Flex Container

Flex Container.- Mecanismo de CSS diseñado para crear un contenedor flexible que facilita la disposición de elementos en una dimensión, ya sea en forma de filas o columnas.

El contenedor se crea mediante display: flex;

```
display: flex;

Propiedades:

•  flex-direction: row  || column || row-reverse || column-reverse;
•  flex-wrap: nowrap || wrap || wrap-reverse;
•  flex-flow: (Combina en una las anteriores)
•  justify-content: flex-start || flex-end || center || space-around ||space-between;
•  align-items: center || flex-start || flex-end || stretch || baseline;
•  align-content: stretch || space-between || space-around || center || etc;
```

CSS Flex Container

Flex Container.- Mecanismo de CSS diseñado para crear un contenedor flexible que facilita la disposición de elementos en una dimensión, ya sea en forma de filas o columnas.

```html
<div class="contenedor">
      <p> …. </p>
      <p> …. </p>
      <p> …. </p>
</div>
```

HTML

```css
.contenedor {
  display: flex;
  …
}

.contenedor > p {
…
}
```

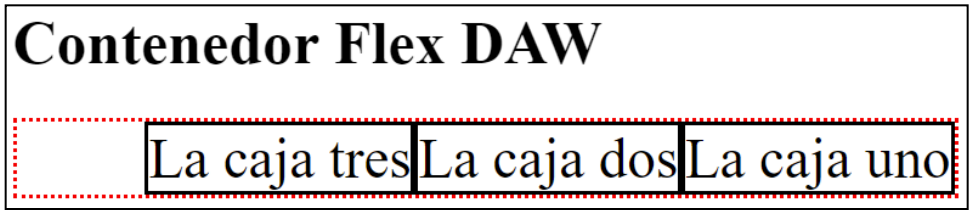
CSS

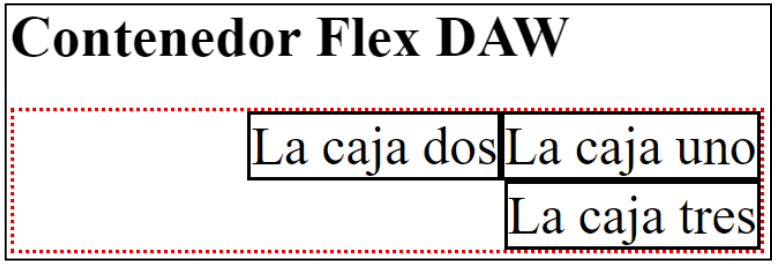
CSS Flex Container

Flex Container.- Mecanismo de CSS diseñado para crear un contenedor flexible que facilita la disposición de elementos en *una dimensión*, ya sea en forma de filas o columnas.

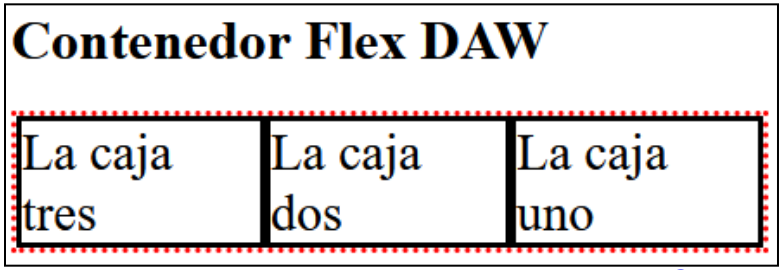
```html
<body>
    <h1>Contenedor Flex DAW</h1>
    <div class="contenedor">
        <div>La caja uno</div>
        <div>La caja dos</div>
        <div>La caja tres</div>
    </div>
</body>
```

```css
.contenedor {
  display: flex;
  flex-direction: row-reverse;
  flex-wrap: wrap;
}
```





wrap



nowrap

CSS Grid

Grid.- Mecanismo de CSS diseñado para facilitar la disposición de elementos en *dos dimensiones*, alineando elementos, tanto en filas como en columnas

El uso de grid se realiza a través de la propiedad *display.*

Para el uso de este mecanismo se definen propiedades tanto para el *contenedor*, como para los ítems que están en su interior.

El contenedor se crea mediante display: grid;

Los ítems se pueden crear en términos de filas/columnas o áreas

CSS Grid

Para el uso de este mecanismo se definen propiedades tanto para el *contenedor*, como para los ítems que están en su interior.

Contenedor:

- Se crea mediante display:grid;
- Se dimensiona mediante grid-template-columns y grid-template-rows
  o mediante grid-template-areas
- Se añaden otras características mediante grid-gap, …

Items:

- Se dimensionan/posicionan mediante grid-column y grid-row.
  o quedan definidos mediante grid-area.

CSS Grid

Contenedor → Mediante grid-template-columns y grid-template-rows
Items → Mediante grid-column y grid-row.

Contenedor:

- Se crea mediante display:grid;
- Se dimensiona mediante grid-template-columns y grid-template-rows
  o mediante grid-template-areas
- Se añaden otras características mediante grid-gap, …

Items:

- Se dimensionan/posicionan mediante grid-column y grid-row.
  o quedan definidos mediante grid-area.

CSS Grid

Contenedor → Mediante grid-template-columns y grid-template-rows
Items → Mediante grid-column y grid-row.

Para el uso de este mecanismo se definen propiedades tanto para el *contenedor*, como para los ítems que están en su interior.

CSS Grid

Contenedor → Mediante grid-template-columns y grid-template-rows
Items → Mediante grid-column y grid-row.

```html
<div class="contenedor">
    <div class="box1">Uno</div>
    <div class="box2">Dos</div>
    <div class="box3">Tres</div>
</div>
```

HTML

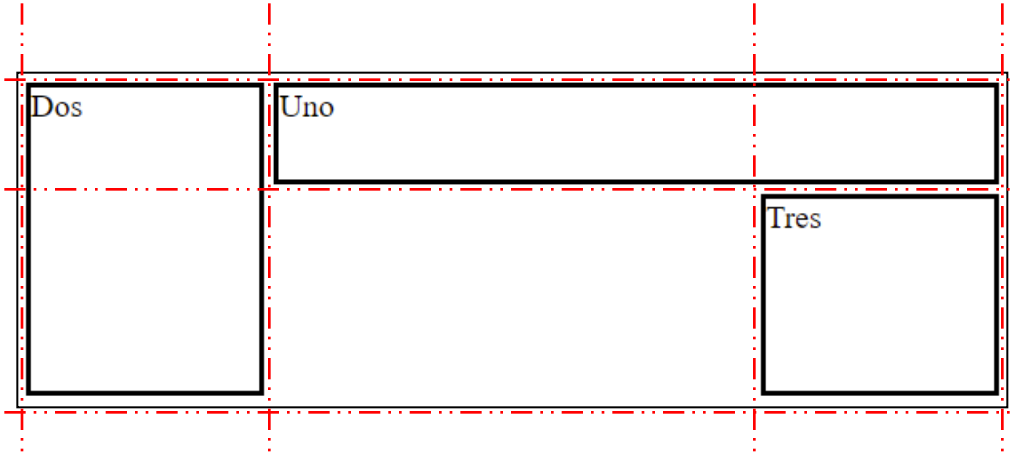
```css
.contenedor {
    display: grid;
    grid-template-columns: 1fr 2fr 1fr;
    grid-template-rows: 50px 100px;
    grid-gap: 5px;
}

.box1 {
    grid-column: 2 / 4;
    grid-row: 1;
}

.box2 {
    grid-column: 1;
    grid-row: 1 / 3;
}

.box3 {
    grid-column: 3;
    grid-row: 2;
}
```

CSS



`fr` → *Unidad para dimensionar el tamaño de la rejilla de forma flexible.*

CSS Grid

Contenedor → Mediante grid-template-areas
Items → Mediante grid-area

Contenedor:

- Se crea mediante display:grid;
- Se dimensiona mediante grid-template-columns y grid-template-rows
  o mediante grid-template-areas
- Se añaden otras características mediante grid-gap, …

Items:

- Se dimensionan/posicionan mediante grid-column y grid-row.
  o quedan definidos mediante grid-area.

CSS Grid

Contenedor → Mediante grid-template-areas
Items → Mediante grid-area

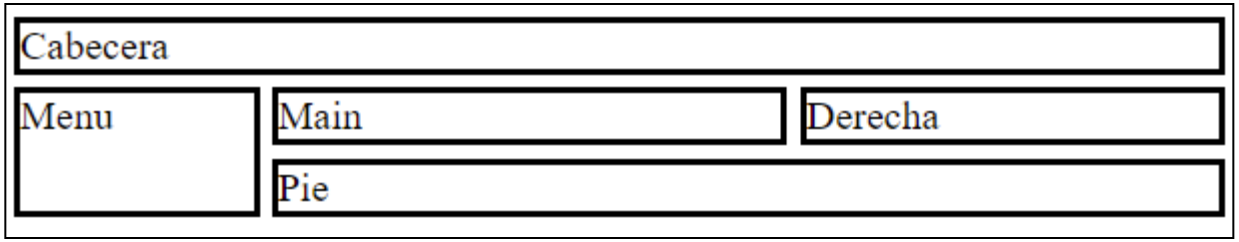
```html
<div class="contenedor">
  <div class="item1">Cabecera</div>
  <div class="item2">Menu</div>
  <div class="item3">Main</div>
  <div class="item4">Derecha</div>
  <div class="item5">Pie</div>
</div>
```

HTML

```css
.item1 { grid-area: header; }
.item2 { grid-area: menu; }
.item3 { grid-area: main; }
.item4 { grid-area: right; }
.item5 { grid-area: footer; }

.contenedor {
  display: grid;
  grid-template-areas:
    'header header header header header header'
    'menu main main main right right'
    'menu footer footer footer footer footer';
  grid-gap: 5px;
}
```

CSS



CSS Librerías/Frameworks

Bibliotecas de reglas CSS *propiedad:valor* predefinidas, apoyados en el atributo *class* de HTML.

Ventajas:

- Facilitan compatibilidad entre navegadores y responsibidad entre dispositivos.
- Simplifican el desarrollo de una aplicación web.
- Garantizan cierto grado de fiabilidad y eficacia.

Inconvenientes:

- Se importa código innecesario, incrementando ancho de banda.
- Se pierde cierto control sobre lo que se está haciendo.
- Se limitan las posibilidades de elección del diseño.

Desarrollo de Aplicaciones Web

CSS Layouts y Responsividad

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Pablo García Tahoces